

Navigation World Models 프로젝트 강의

Text-Conditioned Lightweight NWM / RECON Fork

이 PPT 의 목표는 코드를 바로 실행하기 전에 전체 시스템의 목적과 흐름을 먼저 머릿속에 그리는 것입니다 .

Course Mode

31 slides

개념 → 코드 → 실험 → 실습 순서

Core Model

CDiT

Conditional Diffusion Transformer

Main Dataset

RECON

Trajectory frames + pose/action

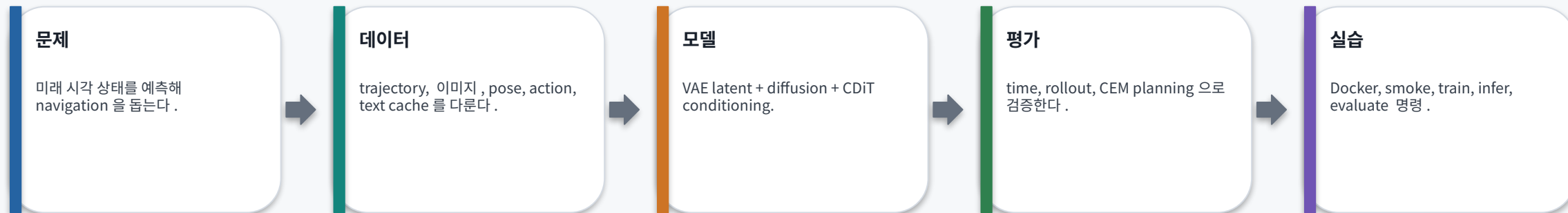
Current Variant

Text

Qwen captions + CLIP embeddings

```
repo: /home/kun/kun_ssd/nwm
outputs: artifacts/decks/nwm_course/
```

수업 로드맵



학습 목표 1

이 repo 에서 어떤 파일이 무엇을 담당하는지 설명할 수 있다 .

학습 목표 2

CDiT world model 이 무엇을 예측하고 어떤 조건을 쓰는지 설명할 수 있다 .

학습 목표 3

평가 결과에서 LPIPS/DreamSim/FID/ATE/RPE 가 의미하는 바를 읽을 수 있다 .

프로젝트 한 줄 정의

Navigation World Model 은 현재 관측과 행동 후보를 조건으로 가까운 미래의 시각 상태를 생성하는 모델입니다 .

입력

과거 context frames
상대 이동 / 회전 action
미래 시간 offset
선택적 text embedding

출력

미래 프레임 또는 rollout
VAE latent 를 거쳐 pixel image 로
복원
후속 평가 / 계획에 사용

학습

정책을 직접 학습하지 않음
행동 조건부 미래 latent 의
denoising 을 학습

계획

후보 action 을 샘플링하고
world model rollout 을 goal image
와 비교해 고름

핵심 질문 : “ 이 행동을 하면 몇 초 뒤 내 카메라가 무엇을 볼까 ?”

이 프로젝트에서 우리가 추가로 묻는 질문

저해상도 또는 경량 모델에서는 이미지에서 길, 벽, 통로, 장애물의 의미가 흐려질 수 있다.

VLM caption 을 offline 으로 만들고 CLIP embedding 으로 cache 하면 semantic prior 로 쓸 수 있다.

텍스트는 명령이 아니라 장면의 의미를 보완하는 조건 신호다.

검증은 frame prediction, rollout, planning 성능으로 나눠서 본다.

주의

텍스트 조건을 넣는다고 planning policy 를 학습하는 것은 아닙니다. 모델 학습은 여전히 future image/latent prediction 이고, CEM 은 평가 시 후보 행동을 고르는 planner 입니다.



Repository Architecture

```

└─ scripts/      nwm/
                  실행 진입점 : train, infer, evaluate, plan_eval
└─ data/         └─ src/      실제 Python package
    │             │
    │             └─ dataset, trajectory/action/image transform
    │             │
    │             └─ models/   CDiT backbone, checkpoint, VAE loader
    │                 │
    │                 └─ diffusion/ DDPM / respacing
    │                     │
    │                     └─ evaluation/inference, metrics, planning
    │                         │
    │                         └─ features/text/ text cache utilities
    │
    └─ configs/    experiment/data/evaluation YAML
    └─ data/splits/ train/test/eval split metadata
    └─ datasets/    raw and derived datasets
    └─ weights/     checkpoints, pretrained, cache
    └─ logs/        train/runtime logs
└─ artifacts/     eval outputs, summaries, decks, figures
  
```

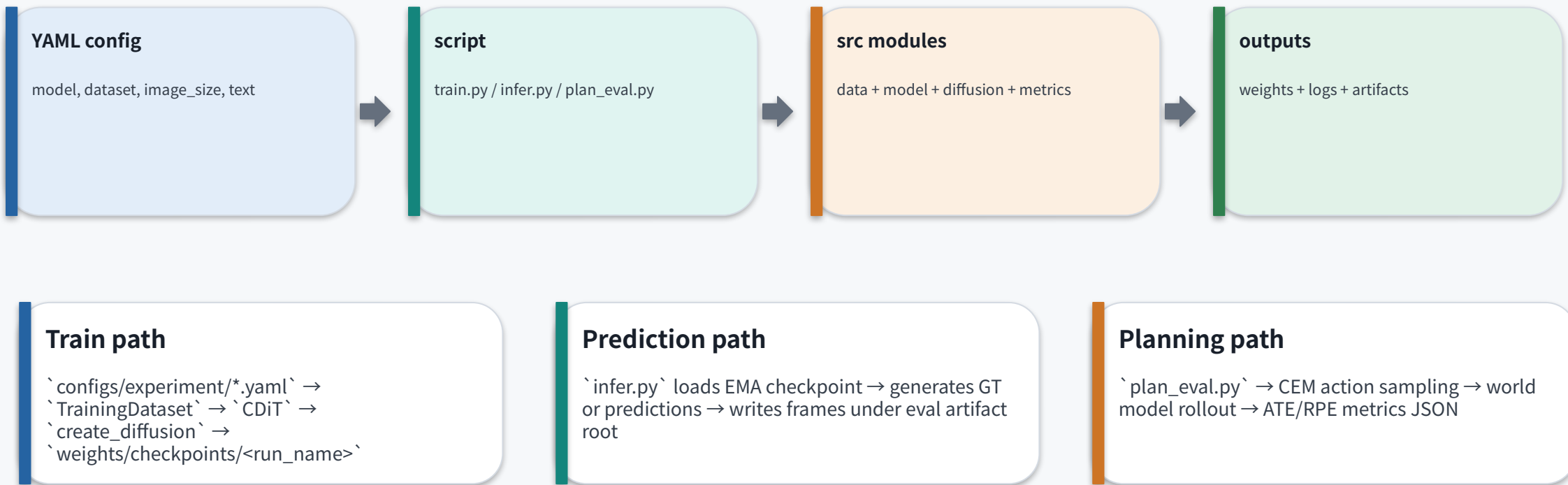
읽는 순서

1. README / docs/project_structure
2. configs/experiment/*.yaml
3. scripts/train.py
4. src/models/backbones/cdit.py
5. scripts/infer.py / evaluate.py / plan_eval.py

중요한 구분

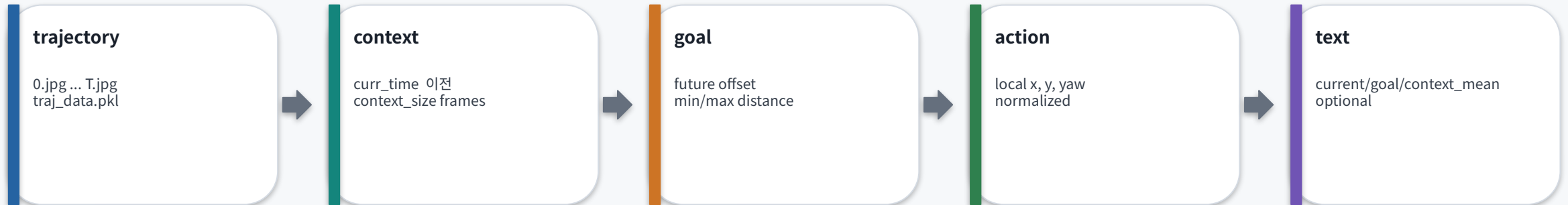
`models/` 폴더가 아니라 `src/models/backbones/cdit.py` 가 실제 모델 코드입니다. 루트의 `diffusion/` 은 stale cache 이고, 실제 구현은 `src/diffusion/` 입니다.

Runtime Architecture



핵심 패턴 : config 는 실험 정의 , scripts 는 실행 버튼 , src 는 재사용 가능한 구현입니다 .

데이터 단위 : trajectory sample



학습 샘플

`obs\_image`: context frames + goal frames
 `goal\_pos`: local coordinate action target
 `rel\_time`: goal offset / 128
 `text\_emb`: optional CLIP vector

데이터셋 코드

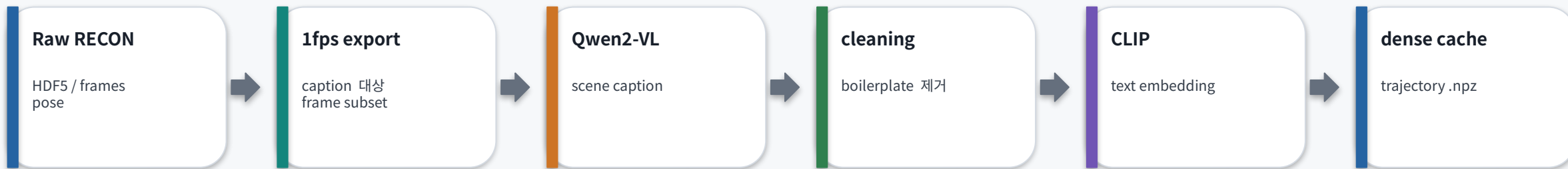
`src/data/datasets/base\_dataset.py`
 index 생성, trajectory 로드, action 계산, text cache
 lookup

 `src/data/datasets/train\_dataset.py`
 random goal offset sampling

평가 데이터

time/rollout/planning 은 predefined index 를 사용해
 같은 샘플 순서로 비교합니다.
 예: `data/splits/recon/test/navigation\_eval.pkl`

RECON + Text Pipeline



`scripts/preprocess/recon/`: RECON export/render helpers

`scripts/preprocess/text/`: Qwen caption, cleaning, CLIP embedding, dense alignment

`datasets/derived/phase1\_text\_embeds\_dense/recon\_all\_raw\_rel`: 학습 시 읽는 dense text cache

`text\_conditioning.enabled: True` 이면 dataset loader 가 text embedding 을 batch 에 추가

왜 offline 인가 ?

학습 / 평가 루프 안에서 VLM 을 돌리면 너무 느립니다 .
caption 과 embedding 을 미리 만들어 cache 하면 기존 NWM 실행 속도와 구조를 크게 깨지 않고 semantic signal 을 추가할 수 있습니다 .

무엇을 학습하는가

학습 목표

```
given: context frames, action(x,y,yaw), relative time,  
      optional text  
      target: future frame latent z_goal  
      diffusion: add noise to z_goal at timestep t  
model: predict denoising target conditioned on inputs  
  
learn  $p(z_{\text{future}} \mid z_{\text{context}}, \text{action}, \text{rel\_time}, \text{text})$ 
```

모델이 직접 배우는 것

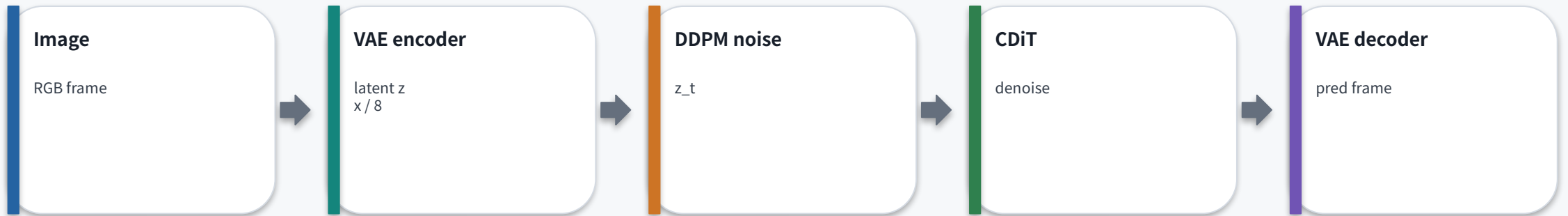
미래 이미지 latent 의 denoising
관측과 행동 사이의 시각적 dynamics
시간 offset 에 따른 변화

모델이 직접 배우지 않는 것

최적 policy
reward function
장기 안전성 보장
CEM hyperparameter 자체

즉, world model 은 “행동의 결과를 상상하는 모델” 이고 planner 는 그 상상을 이용해 행동을 고릅니다.

Latent Diffusion 구조



VAE

``load_vae()`` 로 StabilityAI VAE 를 로드합니다.
이미지는 latent 로 인코딩되고 ``0.18215`` scaling 을 적용합니다.

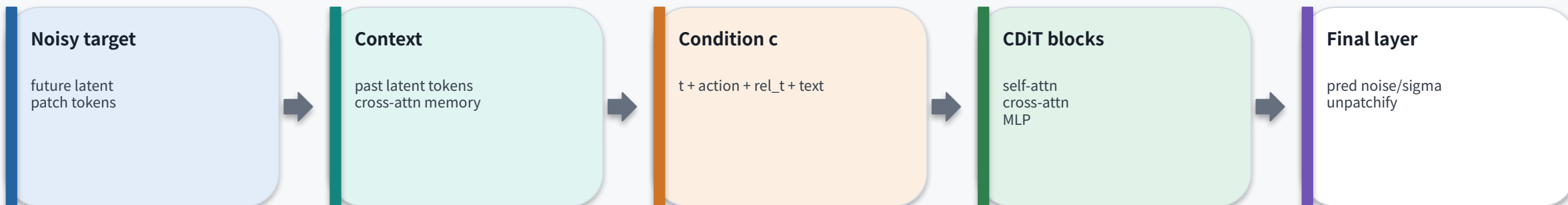
Diffusion

``create_diffusion()`` 은 학습 시 1000 step DDPM, 추론 / 평가에서는 respacing 된 sampling step 을 사용합니다.

장점

픽셀 공간보다 latent 공간이 작아서 학습 / 추론 비용이 줄고, CDiT 가 spatial patch token 을 다루기 쉬워집니다.

CDiT 전체 아키텍처



Model variants

CDiT-S/2: 12 layers, 384 hidden, 6 heads
 CDiT-B/2: 12 layers, 768 hidden, 12 heads
 CDiT-L/2: 24 layers, 1024 hidden, 16 heads
 CDiT-XL/2: 28 layers, 1152 hidden, 16 heads

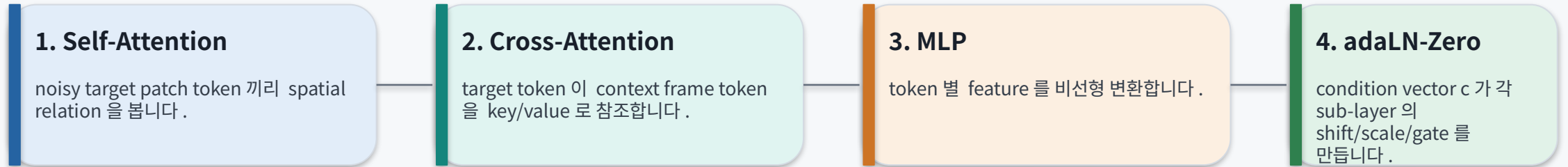
핵심 conditioning

`TimestepEmbedder`: diffusion timestep
 `ActionEmbedder`: x/y/yaw 를 각각 embedding
 `time\_embedder`: relative future offset
 `text\_proj`: CLIP text vector → hidden size

출력

`learn\_sigma=True` 이면 output channel 은 latent channel 의 2 배입니다.
 final layer 후 unpatchify 하여 diffusion loss 에 들어갈 prediction 을 만듭니다.

CDiTBlock 내부



block input

```

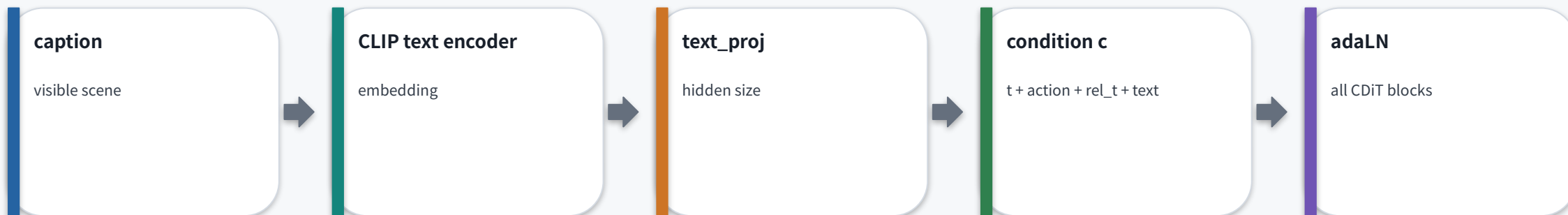
x      = noisy target tokens
x_cond = context frame tokens
c      = t_embed + action_embed + rel_time_embed (+
          text_proj)
  
```

block output

```

x = x + gated self-attn
x = x + gated cross-attn(x_cond)
x = x + gated MLP
  
```

Text Conditioning 은 어디에 들어가나



선택 가능한 source

``current``: 현재 프레임 caption
``goal``: goal frame caption
``context_mean``: context caption 평균

현재 주요 설정

``text_conditioning.enabled: True``
``embedding_root``:
`datasets/derived/.../recon_all_raw_rel``
``condition_source: current``

해석

텍스트는 token sequence 로 cross-attention 되는 방식이 아니라 ,diffusion denoising 의 global condition 을 보정하는 semantic prior 입니다 .

Training Loop



DDP 환경에서 rank 별 DataLoader 를 돌리고 , `DistributedSampler` 로 shuffle 을 맞춥니다 .
context frame 은 conditioning latent 가 되고 , goal frame latent 는 diffusion target 이 됩니다 .
`diffusion.training\_losses(model, x\_start, t, model\_kwargs)` 가 핵심 loss 를 계산합니다 .
EMA 모델은 checkpoint 에 함께 저장되고 , inference/evaluation 에서 EMA weight 를 사용합니다 .
선택적으로 bfloat16, GradScaler, torch.compile, WandB logging 을 사용합니다 .

```
python scripts/train.py --config configs/experiment/nwm_cdit_b_recon_raw_text_dense.yaml --ckpt-every 2000 --eval-every 10000
```

Config 를 읽는 법

```
run_name: nwm_cdit_b_recon_raw_text_dense
  model: CDiT-B/2
  image_size: 224
  context_size: 4
  len_traj_pred: 64
  lr: 8e-5
from_checkpoint: weights/checkpoints/nwm_cdit_b/0100000.pth.tar
  text_conditioning:
    enabled: True
embedding_root: datasets/derived/phase1_text_embeds_dense/recon_all_raw_rel
  condition_source: current
  datasets:
    recon:
data_folder: datasets/recon_raw/recon_release
  train: data/splits/recon/train/
  test: data/splits/recon/test/
```

해석 포인트

`model` 과 `image\_size` 가 architecture 와 latent grid size 를 결정합니다.

`from\_checkpoint` 는 warm-start 여부입니다.

`text\_conditioning` block 이 있으면 model 생성 시 `text\_dim` 이 들어가고 dataset 이 text cache 를 로드합니다.

Inference: time prediction vs rollout

Time prediction

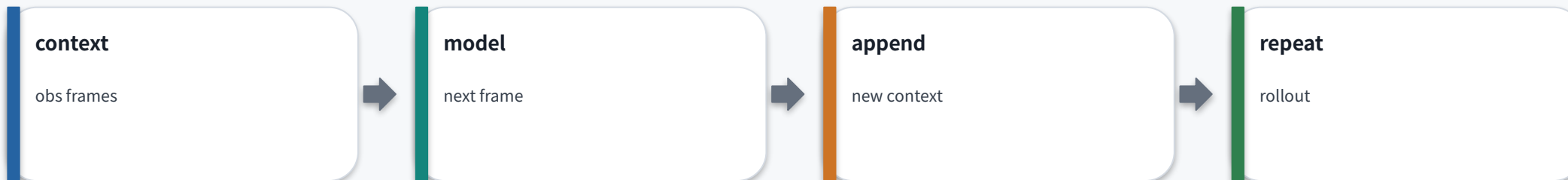
1s, 2s, 4s, 8s, 16s 같은 특정 미래 offset 의 frame 을 한 번에 예측합니다 .

GT 와 직접 비교하기 쉽고 horizon 별 품질을 봅니다 .

Autoregressive rollout

한 step 예측 결과를 다음 context 로 다시 넣어 trajectory 를 이어갑니다 .

누적 오류 , drift, 장기 안정성을 보기에 좋습니다 .



Prediction Metrics

LPIPS

VGG/AlexNet feature 공간에서 perceptual distance 를 계산합니다 . 낮을수록 GT 와 지각적으로 비슷합니다 .

DreamSim

인간 유사도 판단에 맞춘 feature 기반 거리입니다 . 낮을수록 좋습니다 .

FID

예측 이미지 분포와 GT 이미지 분포의 feature statistics 차이를 봅니다 . 낮을수록 좋습니다 .

`infer.py --gt 1` 로 GT frame set 을 먼저 저장합니다 .

`infer.py` prediction mode 로 모델 결과를 같은 구조에 저장합니다 .

`evaluate.py` 가 GT directory 와 experiment directory 를 비교해 JSON metric 을 씁니다 .

```
python scripts/evaluate.py --datasets recon --gt_dir <gt> --exp_dir <pred> --eval_types time,rollout
```

Planning Evaluation: CEM



CEM 은 평가 방법

CEM hyperparameter 를 바꾸는 것은 모델 재학습이 아닙니다. checkpoint 는 그대로 두고, evaluation/planning stage 에서 후보 action 탐색 강도를 바꾸는 것입니다.

Paper-style local setting

```
`N120/K5/rep3/OPT1`  
`num_samples=120`  
`topk=5`  
`num_repeat_eval=3`  
`opt_steps=1`  
`rollout_stride=1`  
`batch_size=1`
```

Trajectory Metrics

ATE

Absolute Trajectory Error
전체 predicted trajectory 가 GT trajectory 와 얼마나 떨어져 있는지 봅니다 . 낮을수록 좋습니다 .

RPE

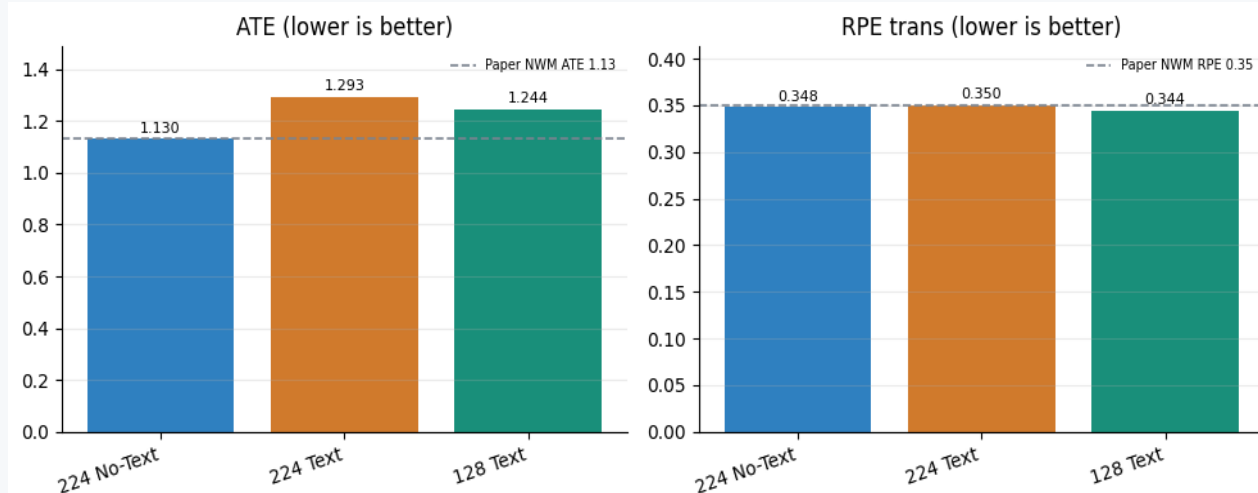
Relative Pose Error
인접 pose 변화의 상대 오차를 봅니다 . local motion consistency 를 반영합니다 .

Pos/Yaw

최종 위치 error 와 yaw error 입니다 . 실제 navigation 에서 goal 근처로 가는지를 보완적으로 확인합니다 .

Planning metric 은 prediction metric 과 다릅니다 . 이미지가 그럴듯해도 action trajectory 가 틀릴 수 있고 , 반대로 일부 이미지 artifact 가 있어도 trajectory 가 맞을 수 있습니다 .

현재 Paper-Style CEM 결과

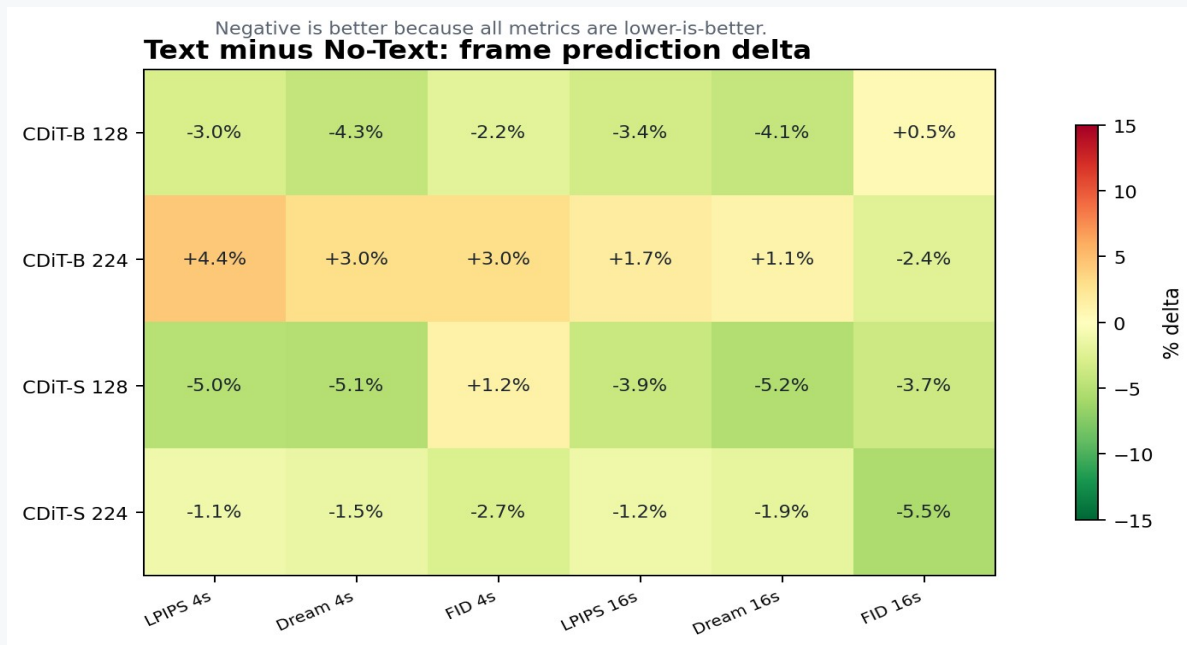


읽는 법

B224 No-Text 는 논문 NWM-only 와 거의 동률입니다 . B224 Text 는 RPE 는 거의 동률 / 소폭 개선이지만 ATE 는 N32 와 논문 기준보다 나쁩니다 . B128 Text 는 N32 대비 ATE/RPE 모두 개선이나 논문 ATE 에는 못 미칩니다 .

Variant	ATE	RPE	ATE vs N32	RPE vs N32	ATE vs Paper
B 224 No-Text	1.130	0.348	-2.72%	-2.09%	-0.01%
B 224 Text	1.293	0.350	+2.56%	-1.17%	+14.42%
B 128 Text	1.244	0.344	-1.28%	-3.99%	+10.12%

Frame Prediction 결과를 읽는 관점



일반적 경향

텍스트 조건은 low-resolution 또는 rollout 처럼 semantic cue 가 사라지기 쉬운 환경에서 LPIPS/DreamSim/FID 를 낮추는 경향을 보였습니다 .

주의

frame prediction 개선이 planning ATE 개선으로 자동 연결되지는 않습니다 . CEM score, action sampler, goal matching, 후반 trajectory outlier 가 planning 결과를 바꿀 수 있습니다 .

Planning 결과 해석 : 왜 metric 이 엇갈리나

ATE 는 전체 trajectory 의 global drift 에 민감합니다 .

RPE 는 local motion 변화의 상대 일관성에 더 가깝습니다 .

B224 Text N120 은 RPE 가 N32 보다 좋아졌지만 , 후반 샘플의 큰 drift 때문에 ATE 가 악화되었습니다 .

따라서 “text 가 navigation 을 무조건 개선한다” 가 아니라 “어떤 metric/setting 에서 개선되는지” 로 읽어야 합니다 .

B224 Text ATE

1.293

N32 대비 +2.56%, paper 대비 +14.42%

B224 Text RPE

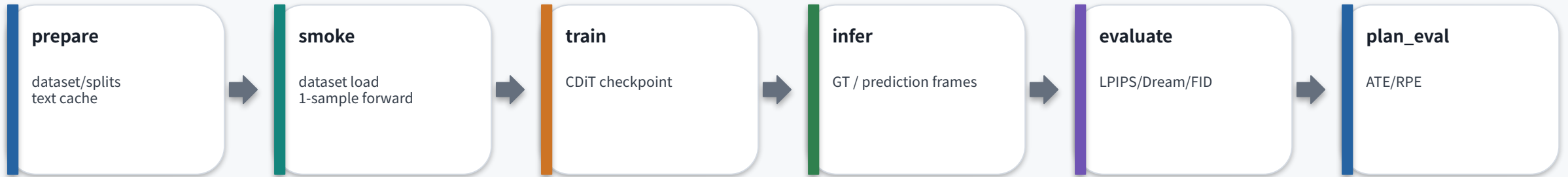
0.350

N32 대비 -1.17%, paper 와 거의 동률

실험 결론

원 논문 CEM 설정으로 바꾸면 evaluation fairness 는 좋아지지만 , 모든 text-conditioned variant 의 ATE 가 자동으로 좋아지는 것은 아닙니다 .

End-to-End 실행 Workflow



```
docker build -t nwm:cu126 .
./scripts/docker/nwm-start.sh
./scripts/docker/nwm-run.sh "python tests/smoke/recon_smoke_test.py --skip-forward"
./scripts/docker/nwm-run.sh "python scripts/train.py --config configs/experiment/nwm_cdit_b_recon_raw_text_dense.yaml"
./scripts/docker/nwm-run.sh "python scripts/infer.py --exp configs/experiment/nwm_cdit_b_recon_raw_text_dense.yaml --datasets recon --eval_type time"
./scripts/docker/nwm-run.sh "python scripts/evaluate.py --datasets recon --gt_dir <gt> --exp_dir <pred>"
./scripts/docker/nwm-run.sh "python scripts/plan_eval.py --exp configs/experiment/nwm_cdit_b_recon_raw_text_dense.yaml --datasets recon --num_samples 120 --topk 5 --num_repeat_eval 3 --opt_steps 1"
```

Smoke Test 와 Debugging

Dataset load

```
`python tests/smoke/recon_smoke_test.py --skip-forward`
```

이미지 , trajectory metadata, action 계산 , text cache shape 를 먼저 확인합니다 .

One forward

```
`python tests/smoke/recon_smoke_test.py --horizon-steps 8`
```

VAE, CDiT forward, diffusion wrapper 가 함께 동작하는지 확인합니다 .

Runtime checks

```
`nvidia-smi`, log tail, result JSON 존재 여부,  
`verify_`*` 스크립트로 산출물 무결성을 확인합니다 .
```

OOM 이면 batch size, num_samples, cem_eval_chunk_size 를 먼저 봅니다 .

checkpoint shape mismatch 는 image_size/model/text_dim 차이일 가능성이 큼니다 .

planning metric 은 JSON 이 생성된 뒤에만 최종값으로 봅니다 . log 의 rolling average 는 중간값입니다 .

Artifacts 와 실험 기록

weights/

학습 checkpoint
`weights/checkpoints/<run\_name>/<step>.pth.tar`

logs/

학습 / 비동기 실행 로그
resolved config 와 runtime 기록

artifacts/

평가 frame, planning JSON, plot, summary, deck

```

├─ bulk/train/      artifacts/
├─ bulk/eval/       training sample dumps
├─ bulk/planning/   GT and prediction frames
├─ summaries/       planning prediction/metric JSONs
├─ profiling/        compact metric CSV/MD/PNG
└─ decks/           runtime/profiling figures
                   presentation outputs

```

실험 재현성

config, checkpoint, dataset split, command, GPU/VRAM, result JSON path 를 같이 기록해야 나중에 결과를 다시 해석할 수 있습니다 .

코드 읽기 로드맵

Week	읽을 파일	질문
1	README.md, docs/project_structure.md	전체 pipeline 을 말로 설명할 수 있나 ?
2	src/data/datasets/*.py	sample 이 어떤 tensor 로 변환되나 ?
3	src/models/backbones/cdit.py	condition vector 가 block 에 어떻게 들어가나 ?
4	scripts/train.py	loss 와 checkpoint 가 어떻게 만들어지나 ?
5	scripts/infer.py, rollout.py	time/rollout prediction 차이는 ?
6	cem_planner.py	CEM 이 action 후보를 어떻게 고르나 ?

읽을 때 항상 “입력 tensor shape → condition → output artifact” 를 추적하세요 .

Lab 1: 데이터셋 한 샘플 해부

`TrainingDataset.\_\_getitem\_\_`에 breakpoint 또는 print를 넣고 한 샘플을 꺼냅니다.

`obs\_image.shape`, `goal\_pos.shape`, `rel\_time`,
`text\_emb.shape`를 기록합니다.

`\_compute\_actions`에서 global pose가 local x/y/yaw로 바뀌는 과정을 따라갑니다.

text cache가 없는 trajectory에서 zero embedding이 들어가는지 확인합니다.

```
python - <<'PY'
from src.config import load_runtime_config
from src.data.transforms.image import build_transform
# config를 읽고 TrainingDataset 하나를 직접 생성해 shape를 출력해보는 연습
PY
```

제출물

샘플 tensor shape 표
한 trajectory의 curr_time/goal_time 예시
텍스트 embedding source 설명

Lab 2: 모델 forward 따라가기

`CDiT.forward(x, t, y, x\_cond, rel\_t, text\_emb)`의 각 인자 shape를 적습니다.

`x\_embedder`, `pos\_embed`, `y\_embedder`, `time\_embedder`, `text\_proj`를 차례로 확인합니다.

`CDiTBlock.forward`에서 self-attn, cross-attn, MLP가 어떤 순서로 적용되는지 도식화합니다.

text_emb=None일 때와 text_emb가 있을 때 condition vector c가 어떻게 달라지는지 비교합니다.

핵심 질문

텍스트는 token attention으로 들어가는가, 아니면 global condition으로 들어가는가?

정답

현재 구현에서는 `text\_proj(text\_emb)`가 `c = t + time + action`에 더해지고, 이 c가 adaLN modulation에 사용됩니다.

Lab 3: 평가 결과 재현

GT frame 생성과 prediction frame 생성을 분리해서 실행합니다.

LPIPS/DreamSim/FID JSON 을 만들고 , horizon 별 metric 이 어떻게 변하는지 plot 합니다 .

planning 은 작은 `--max\_eval\_samples` 로 먼저 smoke 후 full run 으로 확장합니다 .

CEM 설정을 바꿔도 모델 checkpoint 는 바뀌지 않는다는 점을 실험 로그로 확인합니다 .

```
python scripts/plan_eval.py \  
--exp configs/experiment/nwm_cdit_b_recon_raw_text_dense.yaml \  
--datasets recon --ckp 00300000 \  
--num_samples 120 --topk 5 --num_repeat_eval 3 \  
--opt_steps 1 --action_sampler repeat --rollout_stride 1
```

제출물

명령어 , checkpoint, output JSON path, ATE/RPE/Pos/Yaw, GPU 와 runtime 기록

시험에 나올 만한 질문

NWM 은 policy 인가 , world model 인가 ? 왜 그렇게 말할 수 있는가 ?

CDiT 에서 self-attention 과 cross-attention 은 각각 무엇을 보는가 ?

text conditioning 이 모델 architecture 에 들어가는 정확한 위치는 어디인가 ?

time prediction, rollout, planning evaluation 은 무엇이 다르고 왜 모두 필요한가 ?

LPIPS/DreamSim/FID 와 ATE/RPE 는 서로 어떤 종류의 성능을 측정하는가 ?

CEM 설정 변경은 왜 재학습이 아니라 평가 방법 변경인가 ?

B224 Text paper-style CEM 결과에서 ATE 와 RPE 결론이 왜 다르게 나왔는가 ?

핵심 Takeaways

1. 문제

현재 관측과 행동으로 미래 시각 상태를 예측해 navigation planning 을 돕는 world model 입니다 .

2. 방법

VAE latent diffusion + CDiT + action/time/text conditioning 으로 future frame latent 를 denoise 합니다 .

3. 평가

이미지 품질은 LPIPS/DreamSim/FID, navigation 은 CEM planning 의 ATE/RPE 로 봅니다 .

4. 현재 결과

Paper-style CEM full 3/3 완료 . B224 No-Text 는 paper NWM 과 거의 동률 , B128 Text 는 N32 대비 개선 , B224 Text 는 ATE 악화 /RPE 동률권입니다 .

5. 다음 연구 질문

텍스트가 planning ATE 에 손해를 주는 후반 trajectory outlier 를 줄이려면 cost function, text source, ranker, CEM sampler 를 어떻게 바꿔야 하는가 ?

이제 코드를 볼 때는 항상 “ data → latent → condition → denoise → decode → metric ” 흐름으로 추적하면 됩니다 .